PARTE A – Atividade Avaliativa de Manutenção de Software

A) Responsabilidade das classes:

1. Aluno: A classe aluno tem como responsabilidade de criar um molde de como é uma entidade para que se crie um objeto Aluno, com seus devidos atributos, sendo eles o ra, o nome, o e-mail e se ele é adimplente.
2. Disciplina: Da mesma forma que a classe aluno, a classe disciplina tem como principal responsabilidade criar um molde de como obrigatoriamente deve ser uma disciplina, para que, sejam criadas de forma concisa e correta os objetos da classe.
3. Matricula: A classe matricula assim como as outras classes tem como objetivo principal criar um modelo padrão para a criação das matriculas.
4. MatriculaRepository: ele implementa uma validação, que gerencia o acesso aos dados da matricula, seu objetivo principal é de criar uma camada de abstração para a persistência dos dados. Ele basicamente verifica se um aluno especificado pelo ra já está matriculado em alguma disciplina, caso seja, ele retorna falso, caso não seja, ele cria uma matricula.
5. PagamentoService: Aqui reside a lógica de pagamento da aplicação.
6. NotificadorEmail: Aqui reside a lógica de notificações ao e-mail do aluno.
7. MatriculaService: Aqui reside a lógica de matricula de um aluno, ele faz algumas verificações dentro da classe, ele tem a responsabilidade de verificar se o aluno existe, se a disciplina existe, se o aluno esta matriculado em alguma disciplina, se a disciplina tem vagas, se possui documentação pendente, verifica se o aluno é adimplente ou não e cobra o pagamento.

B) Fluxo do método processarMatricula

O método **processarMatricula** inicia validando a existência do aluno e da disciplina, verificando duplicidades no repositório e a disponibilidade de vagas. Após validar os documentos, o sistema calcula o valor final aplicando 50% de desconto para bolsistas e uma taxa de 20 reais caso seja período de ajuste. Na sequência, a camada financeira checa a adimplência e processa a cobrança via gateway de pagamento, retornando erro se recusado. Por fim, se aprovado, o sistema persiste a nova matrícula, dispara um e-mail de confirmação e atualiza o estado da disciplina reduzindo uma vaga.

C) Fatores de dificuldade de compreensão do método

1. **Alta complexidade ciclomática**: Existem muitos níveis de if, o que torna a compreensão do código ser mais difícil de se compreender. Se precisarmos adicionar uma nova validação (por exemplo, verificar pré-requisitos da disciplina), terá que criar mais um nível de recuo ou mexer em uma estrutura de if/else já saturada.
2. **Violação do Princípio de Responsabilidade Única: O método tenta fazer "tudo ao mesmo tempo agora".** Ele é responsável por: Validar integridade de dados (null checks). Validar regras de negócio (vagas, documentos, duplicidade). Calcular o preço da matrícula (lógica financeira). Processar o pagamento. Persistir dados e disparar notificações (e-mail).